

蚌埠学院

课程设计

软件系统设计与体系结

课程名称

构综合实训

个人财务管理与可视化看

题目名称

板

专业班级

2023 级软件工程 2 班

学生姓名

姜萱萱 蓝思钰 刘晓盼

王晨 朱婷婷

学 号

52302042008 52302042011 52302042019

52302042035 52302042055

指导教师

王友

二〇二六年九月

蚌埠学院计算机与信息工程学院课程设计任务书

课 程	软件系统设计与体系结构综合实训	班级	2023 级软件工程 2 班			指导教师	王友	
题 目	个人财务管理与可视化看板		完成时间	2026 年 9 月				
主要内容	1. 账户与交易管理：统一管理资产账户与负债账户，支持收入、支出、转账的新增、编辑、删除与查询，实时推导账户余额与净资产。 2. 预算与目标管理：提供月度分类预算、阈值状态提醒，以及储蓄/还债目标的新增、更新与进度跟踪。 3. 报表与分析：六个月现金流、支出分类结构、结余率等可视化看板，帮助用户回答“钱在哪里、钱花到哪里、下一步如何调整”。 4. 数据导入与可追溯：CSV 账单导入、重复识别、导入批次撤销、JSON 导出与数据彻底删除。 5. 体系结构与质量：MVC 分层与观察者模式应用，提供自动化测试、可执行 JAR 与一键运行脚本。							
设计报告要求	1. 封面：（格式附后） 2. 设计任务书 3. 实训报告： (1) 系统总体方案 (2) 设计思路和主要步骤 (3) 各功能模块和流程图 (4) 设计代码 (5) 心得体会和参考资料 说明：学生完成实训后，提交实训报告及软件，要求文字通畅、字迹工整（也可以打印），并装订成册。							
版面要求	1. 题目用黑体三号，段后距 18 磅（或 1 行），居中对齐； 2. 标题用黑体四号，段前、段后距 6 磅（或 0.3 行）； 3. 正文用小四号宋体，行距为固定值 22 磅； 4. 标题按“一”、“(-)”、“1”、“(1)”顺序编号。							
上机时间安排		一	二	三	四	五	六	日
指导时间地点								

分工协作说明

课题名称	学生姓名	学号	所做的工作
个人财务管理与可视化看板	姜萱萱	52302042008	系统架构设计与代码实现、自动化测试
	蓝思钰	52302042011	需求分析与产品设计文档整理
	刘晓盼	52302042019	数据库设计与技术设计文档整理
	王晨	52302042035	测试用例整理与质量验证文档
	朱婷婷	52302042055	实训报告排版与答辩材料整理
	最后总结调试。由所有组员共同完成		

目 录

蚌埠学院计算机与信息工程学院课程设计任务书	0
个人财务管理与可视化看板	1
一 引言	1
二 系统设计	1
(一) 问题描述	1
(二) 系统总体方案	2
(三) 设计思路和主要步骤	3
(四) 各功能模块和流程图	4
三 系统实现	6
(一) 项目的实现环境	6
(二) 项目的功能实现	6
(三) 数据导入与可追溯性	8
(四) 打包与运行	8
(五) 测试与质量保证	8
四 总结	9
五 参考文献	9
附 程序设计代码	10

个人财务管理与可视化看板

一 引言

个人财务数据通常分散在银行卡、第三方支付、现金和信用账户中。传统记账工具容易停留在流水堆积层面，用户仍然难以快速回答“钱在哪里、钱花到哪里、下一步如何调整”。本项目设计并实现了一套本地优先的个人财务管理与可视化看板，面向拥有多个支付账户、希望建立预算与储蓄习惯的个人用户，围绕账户、交易、预算、目标和报表形成完整业务闭环。

系统以交易记录为唯一可追溯事实来源，支持收入、支出和账户间转账的录入，根据可追溯交易实时推导账户余额、净资产、月度现金流和预算使用状态；同时提供 CSV 账单导入、重复识别、批次撤销、目标进度管理、原始数据导出和彻底删除。界面在桌面端与移动端均可操作，金额支持一键隐藏。

系统采用 MVC 分层架构和观察者模式。Model 集中封装金额、余额与报表计算规则，Controller 负责交互编排并订阅状态事件，View 根据只读快照渲染响应式界面。金额统一以整数分存储，Java 17 本地服务通过 REST API 把数据规范化写入 SQLite；项目提供 39 项 Node 自动化测试、5 项 JUnit 集成测试、真实浏览器回归脚本、可执行胖 JAR 和一键启动脚本，覆盖课程实训的设计、实现、测试、数据库与交付要求。

二 系统设计

（一）问题描述

传统个人记账方式的主要矛盾在于：一是数据分散，银行卡、第三方支付、现金和信用账户余额无法统一查看，用户难以快速回答“钱在哪里”；二是流水堆积，仅记录收支明细而不提供预算节奏、现金流趋势和分类结构，用户难以回答“钱花到哪里、下一步如何调整”；三是数据易丢失，本地表格或纸质记录缺乏可追溯性，导入的账单也难以撤销和管理。

针对上述问题，本系统以交易记录作为唯一事实来源，通过统一账户视图、预算节奏、目标进度和趋势报表把记录转化为可执行信息；通过 CSV 导入、重复识别与批次撤销保证数据治理能力；通过 MVC 分层和观察者模式保证体系结

构清晰、可测试、可演进。

(二) 系统总体方案

1. 主要功能

模块	核心功能	完成情况
财务概览	净资产、收支、结余率、资金轨迹、最近交易	已完成
交易管理	收入/支出/转账、编辑、删除、筛选与搜索	已完成
账户管理	资产/负债账户、期初余额、实时余额	已完成
预算管理	月度分类预算、阈值状态、删除	已完成
目标管理	储蓄/还债目标、新增、更新、删除	已完成
数据治理	CSV 导入、重复识别、批次撤销、JSON 导出	已完成

2. 技术底座

系统采用 Java 17 与内置 HttpServer 构建本地应用服务器，数据以 SQLite 关系型数据库持久化，前端为原生 JavaScript MVC 单页应用，由 Maven 统一构建为可执行胖 JAR；开发阶段可使用 Node.js 启动开发服务器并以 localStorage 降级持久化。

系统采用 MVC 分层体系结构：表现层由响应式 HTML/CSS 与 DashboardView 组成；控制层由 AppController 统一处理导航、表单、筛选、导入导出和高影响确认；领域层由 FinanceStore 与一组纯计算函数组成；持久化适配器在 JAR 模式下调用 Java REST API 和 SQLite，在 Node 开发模式下回退 localStorage。该结构使业务规则、界面和数据访问解耦。

3. 数据模型

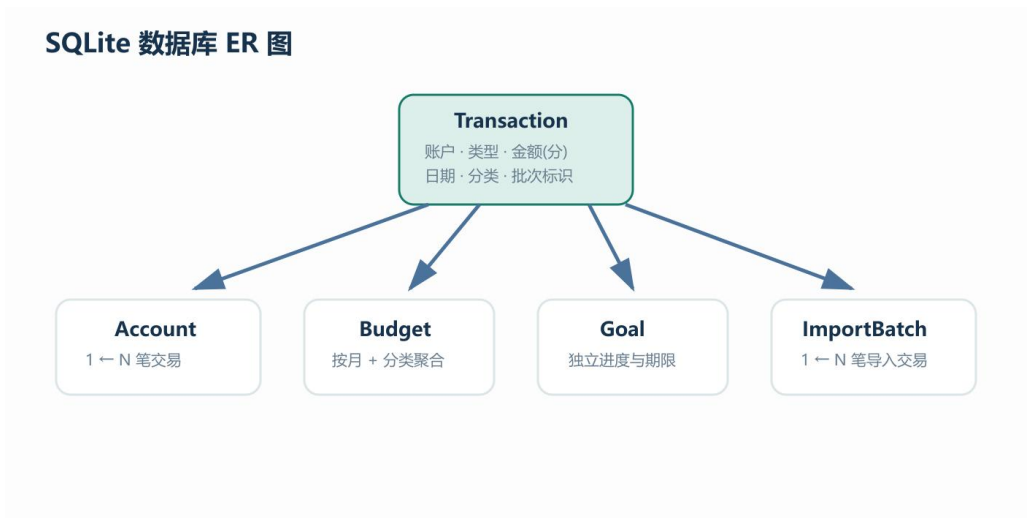


图 1 SQLite 数据库 ER 图

数据库包含 5 张业务表：accounts 保存账户与期初余额；transactions 保存收入、支出、转账事实；budgets 保存分类月度预算；goals 保存储蓄与还债目标；import_batches 保存导入批次及撤销状态。表结构如下：

数据表	主键	用途
accounts	account_id	账户与期初余额
transactions	transaction_id	收入、支出、转账事实
budgets	budget_month + category	分类月度预算
goals	goal_id	储蓄与还债目标
import_batches	batch_id	导入批次及撤销状态

（三）设计思路和主要步骤

整体采用“领域模型驱动 + 分层依赖”的设计思路。先以账户、交易、预算、目标四个领域对象建模，再由 FinanceStore 统一维护状态与事件，计算函数保持纯函数以便单元测试，持久化通过统一 load/save 接口隔离业务操作。模块划分如下：

层次	主要文件	职责
组装入口	src/app.js	创建 Store、View、Controller 并启动
控制层	src/controller/app-controller.js	处理用户事件并订阅 Model 状态
模型层	src/model/finance-model.js	状态、校验、计算与具名事件
视图层	src/view/dashboard-view.js	渲染六个业务页面和可视化
持久化适配	src/data/persistence.js	REST API 优先，localStorage 降级

关键计算规则：

- （1）资产余额 = 期初余额 + 收入 + 转入 - 支出 - 转出。
- （2）负债余额 = 期初未偿金额 + 消费支出 - 还款转入。
- （3）净资产 = 资产账户余额合计 - 负债账户余额合计。
- （4）结余率 = (收入 - 支出) ÷ 收入；收入为零时显示“不适用”。
- （5）预算使用率低于 80% 为正常，达到 80% 为提醒，达到 100% 为超支。
- （6）转账只改变相关账户余额，不进入收入、支出和净现金流统计。

主要实现步骤：先完成领域建模与计算函数，再实现 Store 状态管理与观察者机制，随后编写 View 渲染与 Controller 交互编排，最后接入持久化适配层并补充自动化测试与打包脚本。

（四）各功能模块和流程图

1. 各功能模块

系统按 MVC 划分为视图、控制、模型、持久化四类模块，运行边界限定在本机 127.0.0.1。各模块职责明确：视图模块渲染六个业务页面；控制模块统一处理事件分发与表单校验；模型模块维护状态并发布事件；持久化模块在 JAR 模式下调用 SQLite，在 Node 模式下降级 localStorage。模块间通过只读快照和具名事件通信，架构如图 2 所示。

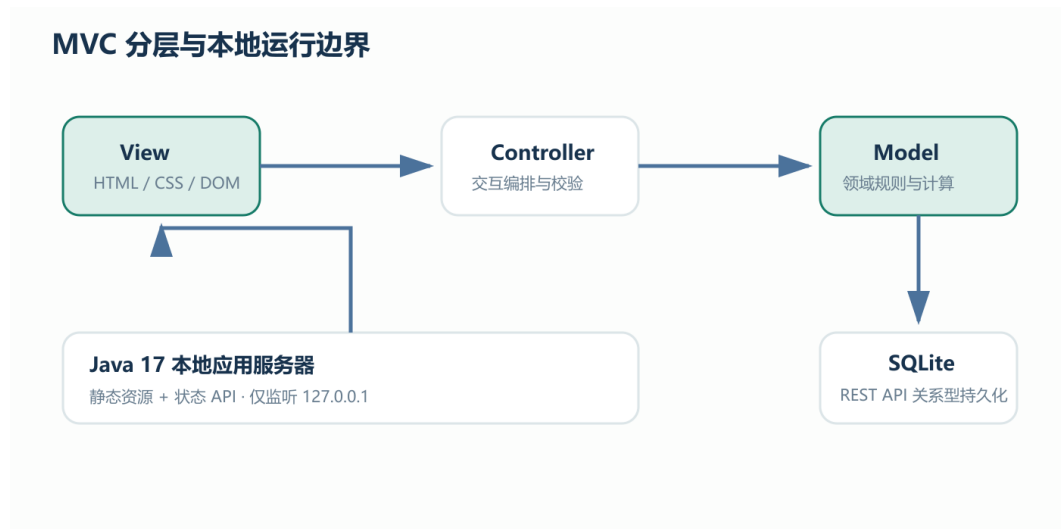


图 2 系统 MVC 分层与本地运行边界

2. 流程图

新增交易流程为：用户打开表单并选择类型 → Controller 读取并规范化字段 → Store 校验金额和账户引用 → Persistence Adapter 调用 PUT /api/state → Java 服务在 SQLite 事务中更新规范化表 → Store 发布 transaction:added 事件 → Controller 取得新快照 → View 同步刷新余额、看板、预算和报表。时序如图 3 所示。

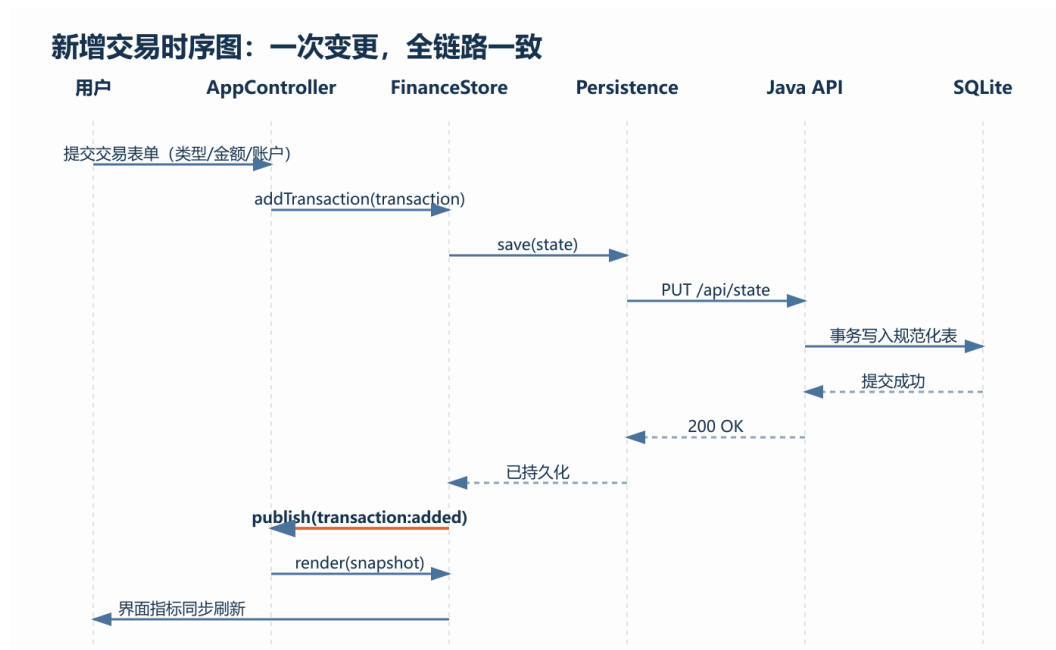


图 3 新增交易时序图

3. 设计模式应用

(1) 观察者模式：FinanceStore 作为主题维护观察者集合，subscribe 用于订阅，所有成功写操作在持久化后发布事件；AppController 订阅状态变更并调用 DashboardView.render。这样，新增交易等业务操作无需知道有哪些界面组件依赖它，降低了模型与 DOM 的耦合。关键代码片段如下：

```

// src/model/finance-model.js — 主题 (Subject)
const observers = new Set();
const publish = (event) => {
  persist(clone(state));
  observers.forEach((observer) => observer(clone(state), event));
};
// 观察者注册与注销接口
subscribe(observer) {
  observers.add(observer);
  return () => observers.delete(observer);
}
// 业务写操作成功后发布具名事件
addTransaction(transaction) {
  validateTransaction(transaction);
  state.transactions.unshift({
    id: transaction.id ??
    createId("transaction"), ...clone(transaction) });
  publish("transaction:added");
}

```

观察者 (Observer)：AppController 启动时订阅 Store，收到事件后立即用最新只读快照刷新全部视图。

```

// src/controller/app-controller.js — 观察者订阅状态变更
start() {
  this.store.subscribe(() => this.render());
}

```

```

this.bindEvents();
this.render();
}

```

观察者模式：一次变更，全部视图一致刷新

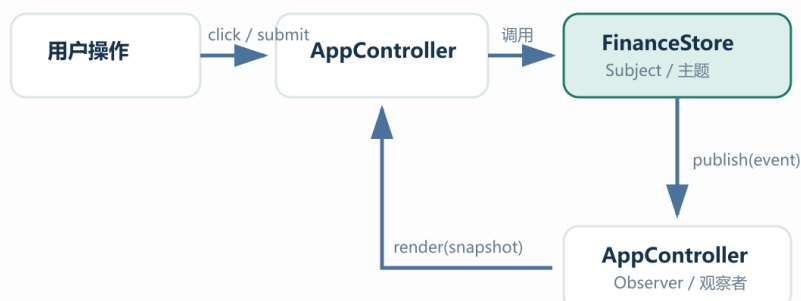


图 4 观察者模式协作过程

（2）策略化计算与适配点：账户余额、月度汇总、预算进度、现金流序列和分类结构均实现为独立纯函数，可以视为可替换的计算策略；持久化通过统一 load/save 接口与业务操作隔离，Java 交付形态使用 SQLite，Node 开发形态使用 localStorage 降级。

三 系统实现

（一）项目的实现环境

后端环境：JDK 17 / Maven 3.8+ / SQLite (sqlite-jdbc 3.43.2.1)；服务基于 JDK 内置 HttpServer，REST API 前缀 /api/state。

前端环境：Node.js 20+（开发模式）/ 现代浏览器；原生 JavaScript + HTML/CSS，无框架依赖。

架构：MVC 分层，通过 RESTful API 通信；程序默认只绑定 127.0.0.1:4173。

（二）项目的功能实现

系统使用深海军蓝、克制绿色和橙色告警构成统一视觉语言。左侧导航在移动端转换为底部六宫格；概览页将净资产和本月现金流置于首要层级；交易、预算和目标均提供明确的创建及维护入口；报表页同时呈现趋势、结构和文字洞察。界面效果如下：

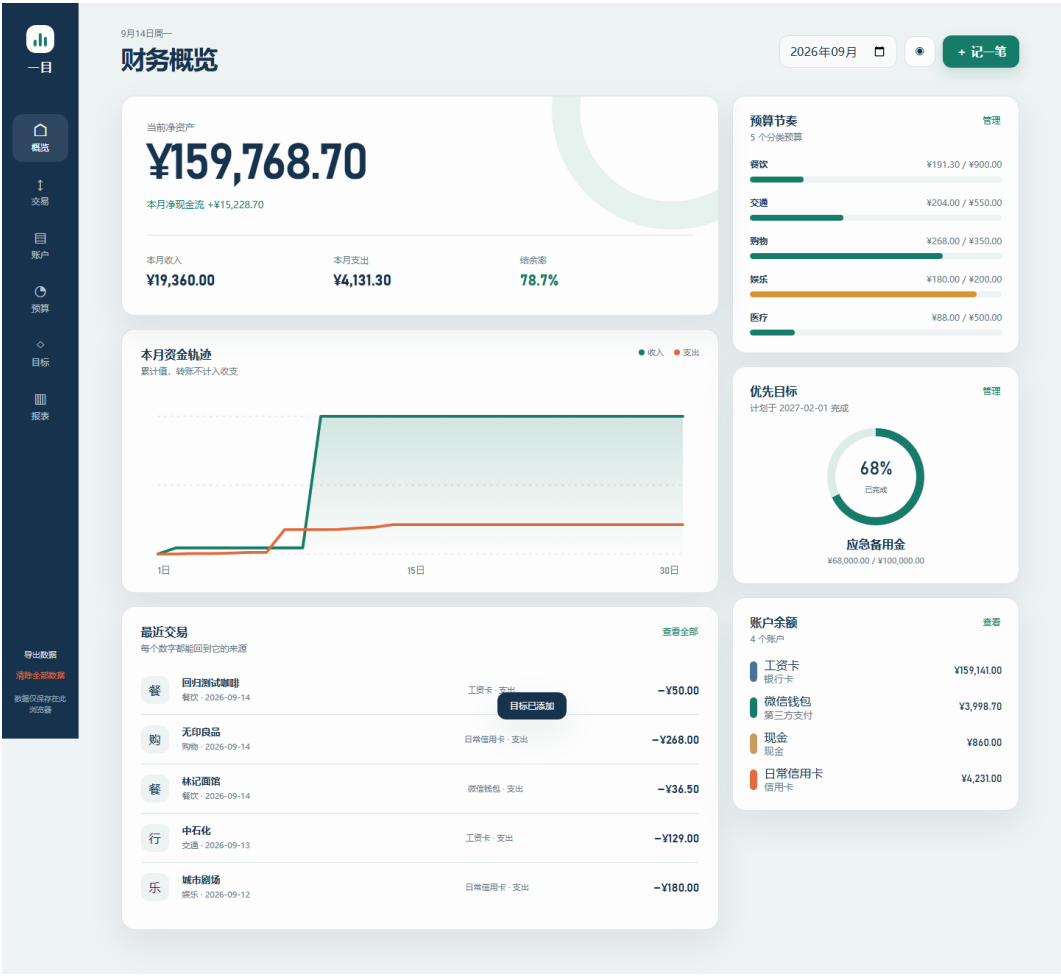


图 5 财务概览桌面端界面



图 6 分析报表界面

（三）数据导入与可追溯性

CSV 解析器支持中文表头和带引号字段，将元转换为整数分。重复指纹由日期、金额、账户和商户组合构成，疑似重复项必须经用户确认后跳过。每次导入形成独立 ImportBatch，并把 batchId 写入相关交易；撤销批次时按该标识精确删除，状态保留为已撤销，便于说明数据变化来源。

（四）打包与运行

Maven 在构建时把 index.html、styles.css 与 src 下的 JavaScript 复制到 JAR 的 web 目录，Main 类使用 JDK 自带 HttpServer 从类路径读取资源。程序默认只绑定 127.0.0.1:4173，运行后浏览器访问该地址即可。run.bat 封装了 java -jar 命令，适合课堂现场演示；无 JAR 的源码包会自动以 Node 模式启动。

（五）测试与质量保证

测试分四个层次进行：Node 领域单元测试覆盖余额、净资产、预算边界、CSV、观察者、目标与报表；JUnit 集成测试覆盖建表、事务持久化、跨重启恢复与接口限制；结构与安全测试覆盖 MVC、页面、路径穿越、隐藏目录与 JAR 配置；真实浏览器回归在交付 JAR 上执行。测试方法与结果如下：

测试层次	覆盖内容	结果
Node 领域单元测试	余额、净资产、预算边界、CSV、观察者、目标、报表	全部通过
JUnit 数据库/API	建表、事务持久化、跨重启恢复、接口限制	全部通过
结构与安全测试	MVC、页面、路径穿越、隐藏目录、JAR 配置	全部通过
浏览器回归	交易编辑、导入撤销、预算、目标、报表、响应式布局	全部通过

关键测试用例如下：

编号	场景	预期结果
T01	资产账户发生收入、支出和转账	余额按规则准确更新
T02	信用卡消费后通过资产账户还款	负债先增加后减少
T03	收入为零时计算结余率	返回不适用，无 Infinity/NaN
T04	CSV 含引号商户并重复导入	正确解析并提示疑似重复
T05	撤销一个导入批次	仅移除该批次交易并重

		算指标
--	--	-----

截至提交版本，Node 自动化测试共 39 项、JUnit 集成测试共 5 项，全部通过；真实 Chromium 回归在交付 JAR 上执行通过，页面无控制台错误。测试命令为 `npm run validate`、`mvn test` 和 `npm run test:browser`。

四 总结

本项目完成了从需求分析、产品设计、体系结构设计、详细设计到编码、测试和部署的完整实训链路。系统不以静态原型结束，而是交付可直接运行的 Java 程序和配套源代码；核心业务闭环、报表分析、数据治理及安全边界均有实现与测试证据。MVC 使关注点清晰分离，观察者模式保证不同视图对同一状态变更保持一致，整数分和纯函数计算提高了财务数据的正确性与可测试性。

在安全性方面，所有写入 innerHTML 的用户字段统一做 HTML 实体编码；CSV 内容上限为 2MB，交易金额必须为正，账户引用必须存在；静态资源只允许 GET/HEAD，状态 API 只允许 GET/PUT；响应包含 CSP、X-Content-Type-Options、X-Frame-Options 和 Referrer-Policy；清空数据、删除交易、撤销导入等高影响操作均需用户确认。

后续可以在保持领域接口稳定的基础上增加登录授权、数据库迁移、可视化字段映射、年度预算和多端同步。本次实训说明了体系结构并非形式化图示，而是直接影响代码可读性、变化成本、测试难度和部署可靠性的工程决策。

五 参考文献

- [1] 软件系统设计与体系结构综合实训指导书，蚌埠学院计算机与信息工程学院。
- [2] Gamma E, Helm R, Johnson R, Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- [3] Fielding R. Architectural Styles and the Design of Network-based Software Architectures. University of California, Irvine, 2000.
- [4] WHATWG. HTML Living Standard. 表单、Dialog、Web Storage 相关规范。

附 程序设计代码

1. finance-model.js —— 领域模型（观察者模式与金额计算）

```
const amountForAccount = (account, transaction) => {
  const amount = transaction.amountCents;
  const isSource = transaction.accountId === account.id;
  const isTarget = transaction.targetAccountId === account.id;
  if (account.kind === "liability") {
    if (transaction.type === "expense" && isSource) return amount;
    if (transaction.type === "income" && isSource) return -amount;
    if (transaction.type === "transfer" && isTarget) return -amount;
    if (transaction.type === "transfer" && isSource) return amount;
    return 0;
  }
  if (transaction.type === "income" && isSource) return amount;
  if (transaction.type === "expense" && isSource) return -amount;
  if (transaction.type === "transfer" && isSource) return -amount;
  if (transaction.type === "transfer" && isTarget) return amount;
  return 0;
};

export const calculateAccountBalance = (account, transactions) =>
  transactions.reduce((balance, t) => balance + amountForAccount(account, t),
    account.openingBalanceCents);

export const calculateSummary = (accounts, transactions, month) => {
  const balances = accounts.map((a) => ({ kind: a.kind, balanceCents:
    calculateAccountBalance(a, transactions) }));
  const assets = balances.filter((b) => b.kind === "asset").reduce((s, b) => s +
    b.balanceCents, 0);
  const liabilities = balances.filter((b) => b.kind === "liability").reduce((s, b)
    => s + b.balanceCents, 0);
  return { assetsCents: assets, liabilitiesCents: liabilities,
    netWorthCents: assets - liabilities, savingsRate: income === 0 ? null : (income
    - expense) / income };
};
```

2. app-controller.js —— 控制器（订阅与事件分发）

```
export class AppController {
  constructor(store, view) { this.store = store; this.view = view; }
  start() {
    this.store.subscribe(() => this.render());
    this.bindEvents();
    this.render();
  }
  render() { this.view.render(this.store.getState(), this.ui); }
  handleClick(event) {
    const actionButton = event.target.closest("[data-action]");
    if (!actionButton) return;
    const actions = {
      "new-transaction": () => this.openTransactionDialog(),
      "edit-transaction": () => this.openTransactionDialog(...),
      "undo-import": () => this.undoImport(actionButton.dataset.id),
      "export-data": () => this.exportData(),
    };
    actions[actionButton.dataset.action]();
  }
}
```

```

    };
    actions[actionButton.dataset.action]?.();
  }
}

```

3. persistence.js —— 持久化适配层（REST API 优先，localStorage 降级）

```

export const createPersistence = ({ fetcher, storage, seedData, storageKey }) => {
  const save = (state) => {
    storage.setItem(storageKey, JSON.stringify(normalizeState(state)));
    const writeServer = async () => {
      try {
        const response = await fetcher("/api/state", { method: "PUT",
          headers: { "Content-Type": "application/json" }, body:
JSON.stringify(snapshot) });
        return response.ok;
      } catch { return false; }
    };
    serverWriteQueue = serverWriteQueue.then(writeServer, writeServer);
    return serverWriteQueue;
  };
  const load = async () => {
    try {
      const response = await fetcher("/api/state", { headers: { Accept:
"application/json" } });
      if (!response.ok) throw new Error("state api unavailable");
      const payload = await response.json();
      if (payload.initialized) return normalizeState(payload.state);
      const initial = normalizeState(seedData);
      await save(initial);
      return initial;
    } catch { return readLocal(); }
  };
  return { load, save };
};

```

4. Main.java 与 FinanceHttpServer.java —— Java 本地服务（入口与 API）

```

// Main.java — 可执行 JAR 入口
public static void main(String[] args) throws IOException, SQLException {
  int port = readPort();
  FinanceHttpServer server = new FinanceHttpServer(readDatabasePath(), port);
  server.start();
  Runtime.getRuntime().addShutdownHook(new Thread(server::close));
  System.out.printf("一目个人财务看板已启动: http://127.0.0.1:%d/%n", port);
}

// FinanceHttpServer.java — 状态 API 路由与安全限制
private void handleStateApi(HttpExchange exchange) throws IOException {
  addSecurityHeaders(exchange.getResponseHeaders());
  switch (exchange.getRequestMethod()) {
    case "GET" -> sendJson(exchange, 200, Map.of(

```

```
        "initialized", repository.isInitialized(), "state",
repository.loadState());
        case "PUT" -> saveState(exchange);
        default -> sendJson(exchange, 405, Map.of("error", "仅支持 GET 和 PUT 请求
"));
    }
}
```